

SEN3006 – Software Architecture

Java Design Pattern Project

GENERAL INFORMATION

- **Applicants' Names:** Abdul Rahman Malak and Mouhammad Houjeirat
 - **Project Title:** Task Management System for Software Development Teams — A Factory Method and Strategy Pattern Implementation
 - **Course:** SEN3006 – Software Architecture
 - **Design Patterns:** Factory Method (Creational) + Strategy (Behavioral)
 - **Submission Date:** June 2026
-

SUMMARY

This project delivers a Task Management System built in pure Java that demonstrates two complementary GoF design patterns — **Factory Method** (creational) and **Strategy** (behavioral) — applied to a realistic software-engineering problem: coordinating heterogeneous work items (bugs, features, documentation) under changing prioritization needs.

- **Originality:** Unlike textbook examples that showcase a single pattern in isolation, this project composes two patterns within one coherent system and shows how they interact through a single coordinator (`TaskManager`) that simultaneously acts as a Factory Method *client* and a Strategy *context*. The system is extensible along two independent axes — new task types and new prioritization algorithms — without modifying existing code.
 - **Methodology:** Factory Method is used for polymorphic task creation; Strategy is used for runtime-swappable prioritization. The design is driven by SOLID principles (SRP, OCP, LSP, ISP, DIP) and validated through seven UML diagrams (class, sequence, use case, activity, state, component, deployment).
 - **Project Management:** Work is structured into five work packages (WP1–WP5): problem definition, design & UML, implementation, testing, and reporting. Each WP has defined deliverables and risk-mitigation strategies.
 - **Expected Impact:** The project demonstrates that high-quality architecture can be achieved without heavy frameworks. It serves as an educational reference for applying GoF patterns and SOLID principles in real Java code, and provides a foundation that can be extended toward a production-grade task tracking tool.
-

1. ORIGINALITY

1.1 Importance and Originality

Problem. Software development teams operate on heterogeneous work items — bug reports, feature requests, and documentation tasks — each with distinct attributes, workflows, and prioritization needs. A bug has a severity level and reproduction steps; a feature has an effort estimate and business value; a documentation task has a document type and a target audience. On top of this, the *right* order in which to work on these items depends on context: crisis mode demands urgent-first sorting, sprint planning demands deadline-first sorting, and a stabilization phase demands severity-first sorting of bugs.

Why current solutions are insufficient. Naive implementations embed task-creation logic in long `if/else` or `switch` blocks and hard-code sorting algorithms inside the manager class. This tightly couples the system to its current task types and strategies: introducing a new task type (e.g., `ResearchTask`) or a new prioritization rule forces modifications across the codebase, violating the Open/Closed Principle and creating a brittle maintenance burden. Existing commercial tools (Jira, Trello, Linear) solve the *user* problem but hide their architecture; they do not serve as a pedagogical reference for students learning design patterns.

Research question. *How can design patterns be composed to build a task management system that is simultaneously extensible in its product hierarchy and flexible in its algorithmic behavior, without sacrificing clarity or introducing external dependencies?*

Originality. The project's original contribution is the demonstration of **double extensibility** through the deliberate composition of Factory Method and Strategy inside a single coordinator class, accompanied by a complete UML model, a working Java implementation, and test sections that explicitly exercise each SOLID principle.

1.2 Aim and Scope

Aim. To design, implement, and evaluate a Task Management System that uses Factory Method and Strategy patterns to achieve an extensible, maintainable architecture in pure Java.

Scope — Included:

- Three task types: Bug, Feature, Documentation (with type-specific fields).
- Three prioritization strategies: Urgent-First, Deadline-First, Severity-First.
- A validated task lifecycle state machine (OPEN, IN_PROGRESS, REVIEW, DONE, BLOCKED).
- A `TaskManager` that coordinates creation, prioritization, filtering, and reporting.
- Seven UML diagrams and a complete test suite in `Main.java`.
- Zero external dependencies — only `java.util` and `java.time`.

Scope — Excluded: persistent storage, multi-user concurrency, GUI, authentication, network services. These are acknowledged as future work.

Expected Outcomes:

1. A fully working Java console application demonstrating both patterns.
2. UML documentation suitable for architectural review.

3. A written report explaining design decisions, trade-offs, and counterfactual analysis.
 4. A reusable educational artifact illustrating SOLID principles in action.
-

2. METHOD

System Architecture

The system follows a layered object-oriented architecture organized into five logical components:

- **Core:** `Task` interface, `AbstractTask` abstract class, `TaskStatus` enum — foundational abstractions.
- **Products:** `BugTask`, `FeatureTask`, `DocumentationTask` — concrete task types.
- **Factories:** `TaskFactory` abstract class and three concrete factories — Factory Method hierarchy.
- **Strategies:** `PriorityStrategy` interface and three concrete strategies — Strategy hierarchy.
- **Coordinator:** `TaskManager` — the central class that acts as client of Factory Method and context of Strategy.

Selected Design Patterns

1. **Factory Method (Creational).** Defers instantiation to subclasses. `TaskFactory` declares `createTask(...)`; `BugTaskFactory`, `FeatureTaskFactory`, and `DocumentationTaskFactory` each override it to produce their specific product with sensible defaults. `TaskFactory` also provides a `createTaskWithDeadline(...)` *template method* layered on top of the factory method.
2. **Strategy (Behavioral).** Encapsulates a family of algorithms behind a common interface. `PriorityStrategy.sort(List<Task>)` is implemented by `UrgentFirstStrategy` (priority descending), `DeadlineFirstStrategy` (deadline ascending, nulls last), and `SeverityFirstStrategy` (two-tier sort: bugs first by severity, then non-bugs by priority). The `TaskManager` holds a `PriorityStrategy` reference and delegates sorting to it; strategies are swappable at runtime via `setPriorityStrategy(...)`.

These two patterns were chosen because they address orthogonal concerns — *object creation* and *algorithmic behavior* — and their composition inside the `TaskManager` illustrates how patterns can coexist naturally.

UML Diagrams

Seven diagrams are produced as PlantUML sources in [docs/uml/](#):

Diagram	File	Purpose
Class	class/task-management-class.puml	Complete static structure — product, creator, strategy hierarchies, and coordinator.
Sequence	sequence/task-creation-sequence.puml	Runtime flow of <code>TaskManager.createTask("BUG", ...)</code> .
Use Case	usecase/task-management-usecase.puml	Developer and Team Lead interactions.
Activity	activity/task-lifecycle-activity.puml	End-to-end workflow from creation to completion.
State	activity/task-state.puml	<code>TaskStatus</code> finite state machine.
Component	class/component-diagram.puml	Module-level dependencies.
Deployment	class/deployment-diagram.puml	JVM runtime environment.

The rendered diagrams follow. The first three (class, sequence, use case) are mandatory; the remaining four (activity, state, component, deployment) are included because the system has a non-trivial task lifecycle and a modular layered structure.

Figure 1 — Class Diagram. Static structure: the `Task / AbstractTask` product hierarchy, the `TaskFactory` creator hierarchy (Factory Method), the `PriorityStrategy` algorithm family (Strategy), and the `TaskManager` coordinator that composes both.

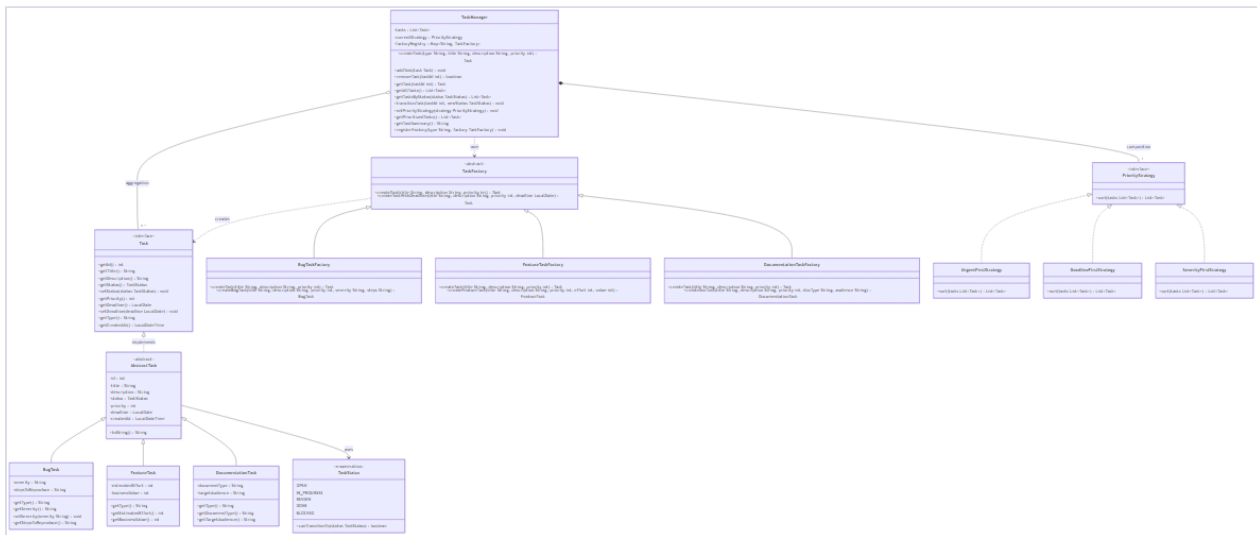


Figure 2 — Sequence Diagram. Runtime interaction for `TaskManager.createTask("BUG", ...)`: factory lookup, delegated instantiation, and registration of the new task.

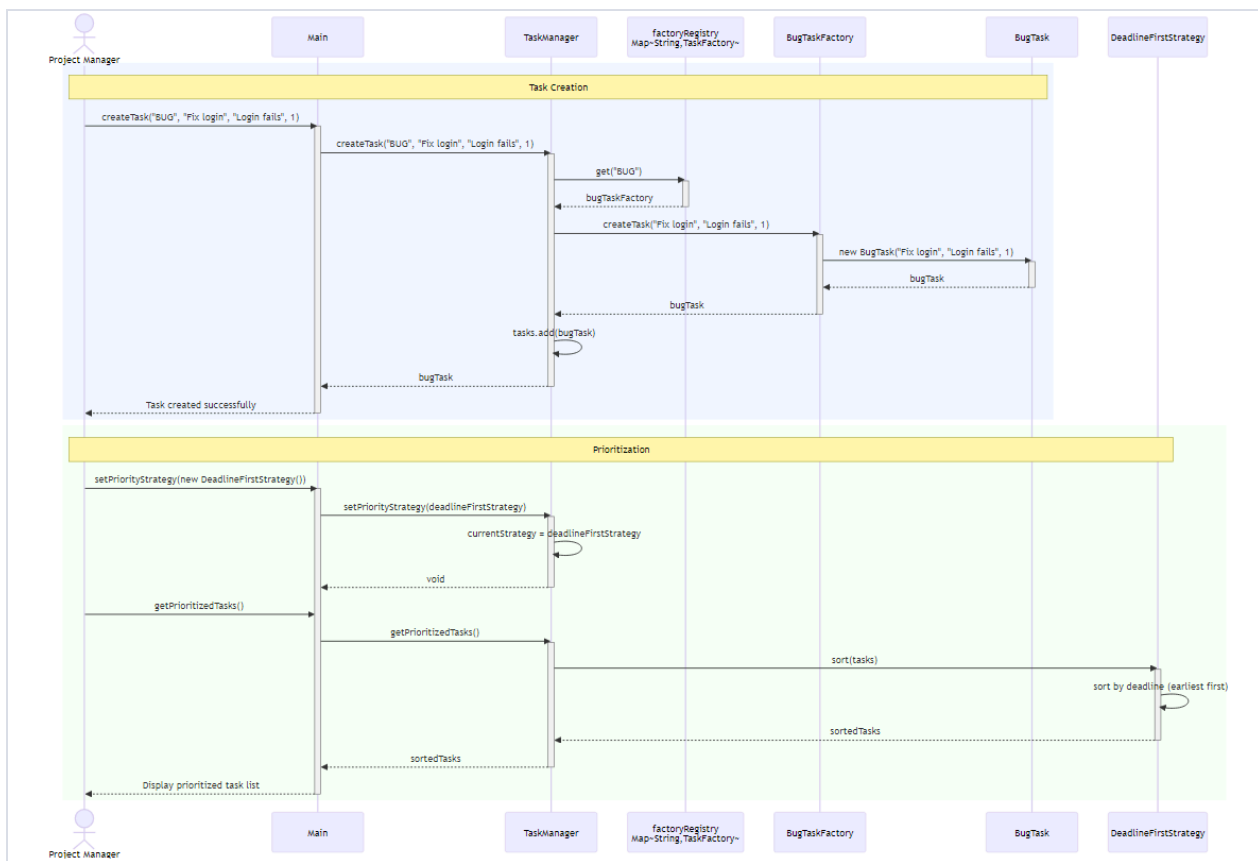


Figure 3 — Use Case Diagram. Developer and Team Lead interactions with the system (create, transition, prioritize, filter, report).

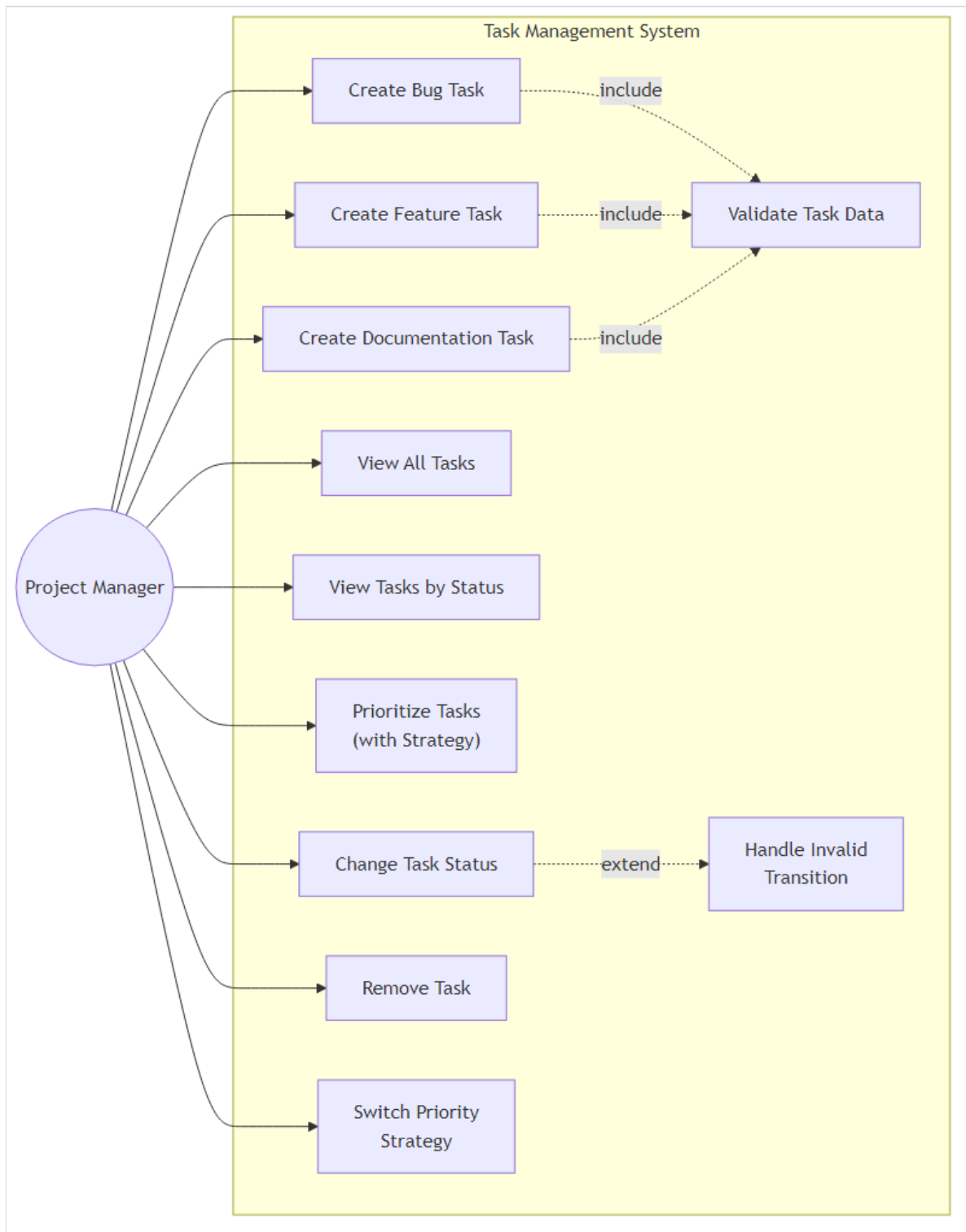


Figure 4 — Activity Diagram. End-to-end workflow of a task from creation through prioritization to completion.

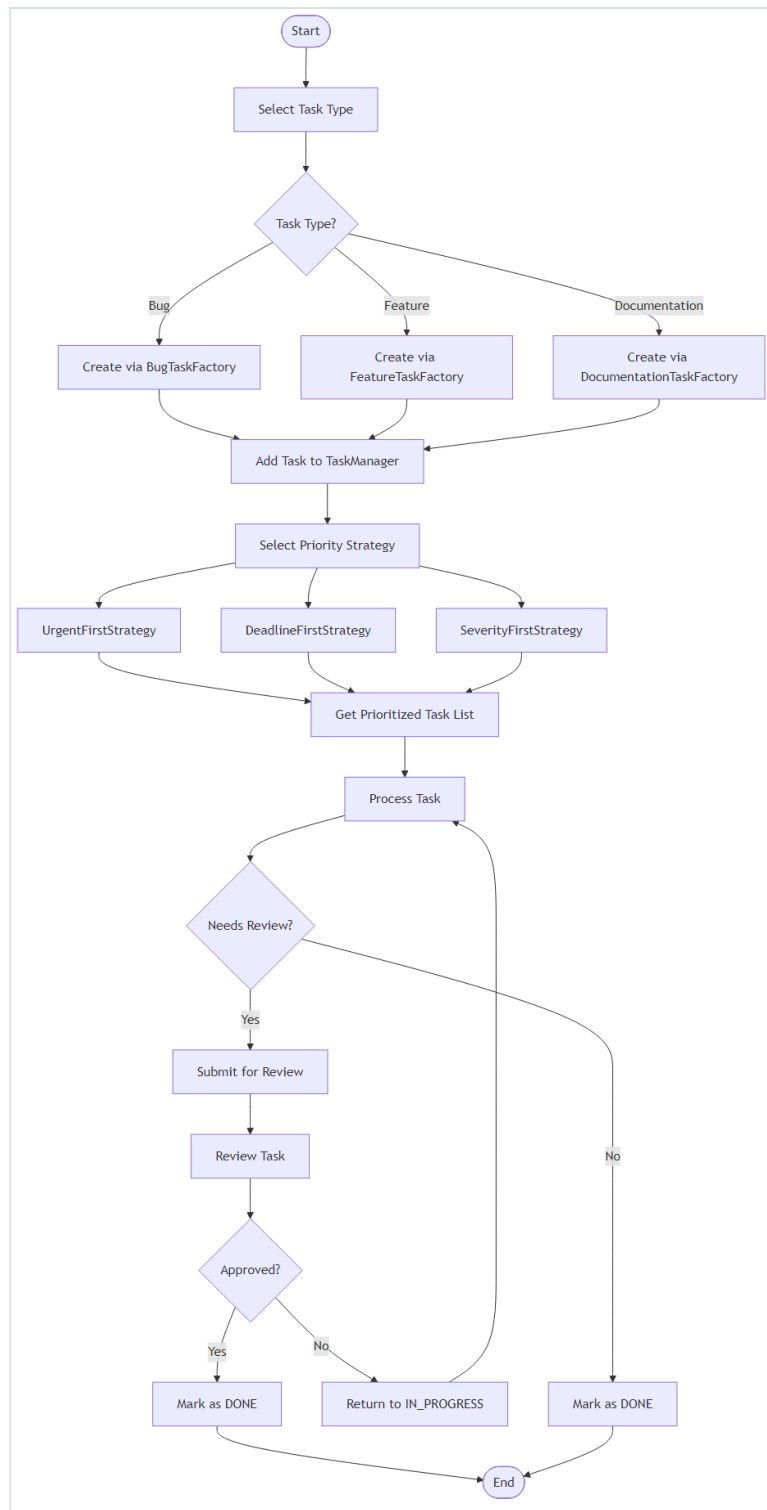


Figure 5 — State Diagram. The `TaskStatus` finite state machine (OPEN, IN_PROGRESS, REVIEW, DONE, BLOCKED) and its permitted transitions.

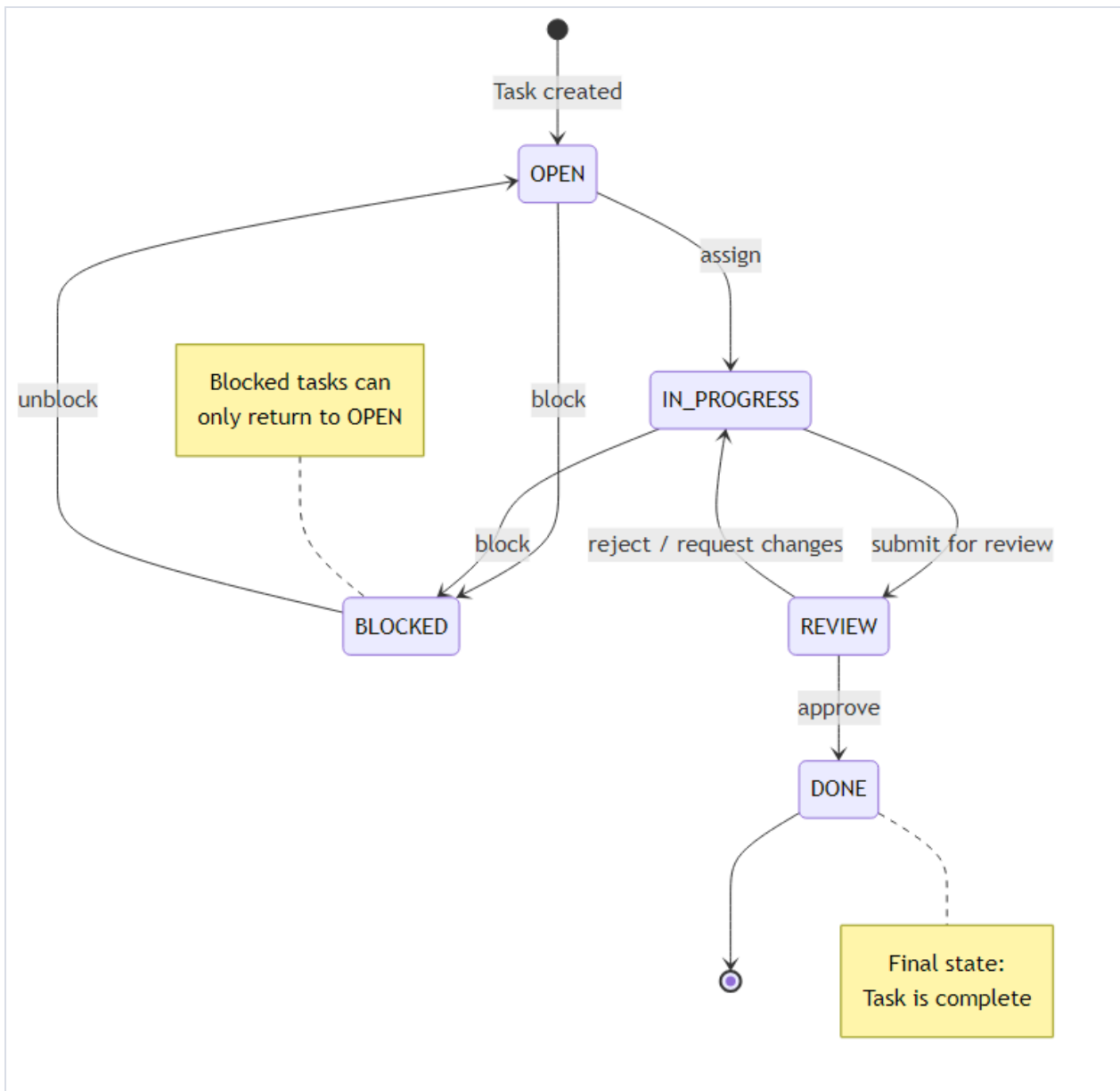


Figure 6 — Component Diagram. Module-level dependencies between the Core, Products, Factories, Strategies, and Coordinator components.

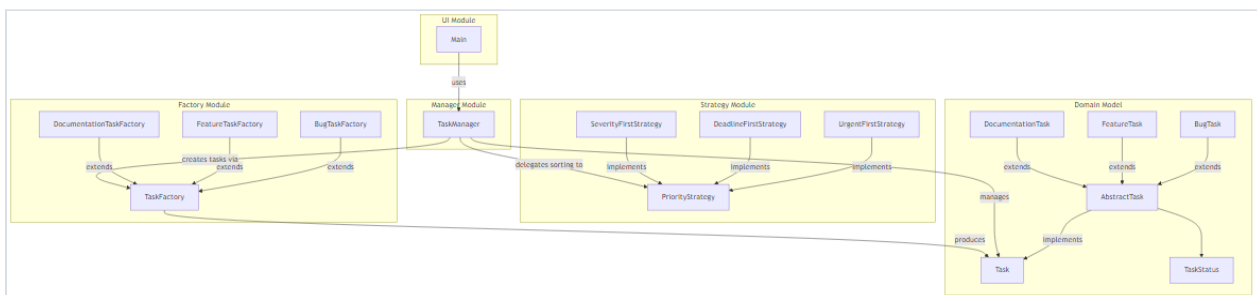
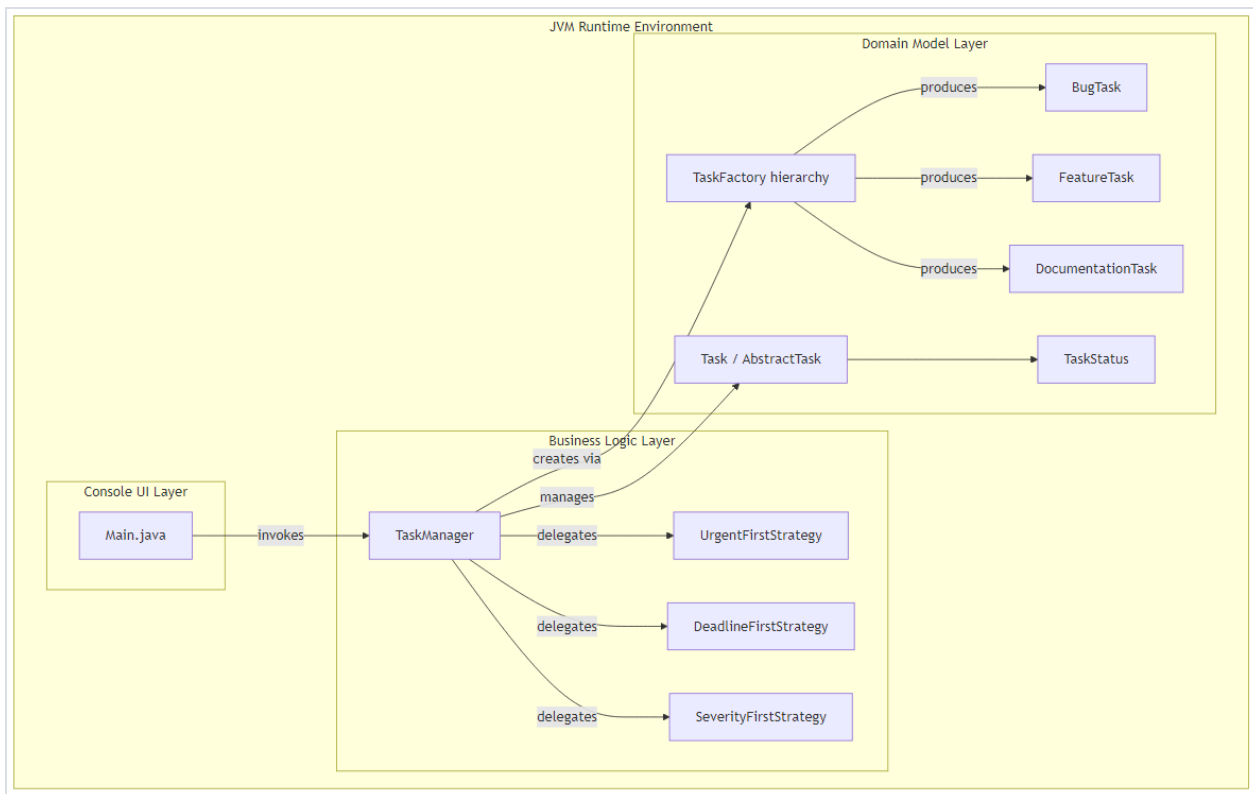


Figure 7 — Deployment Diagram. The single-JVM runtime environment in which the application executes.



Implementation Approach in Java

- **Language level:** Java 8+.
- **Package structure:** single default package (16 source files in `src/main/java/`).
- **Abstraction layering:** interfaces and abstract classes at the top of each hierarchy; concrete implementations at the bottom.
- **Immutability:** strategies return *new* sorted lists without mutating the caller's list.
- **State machine:** enforced by per-constant `allowedTransitions()` overrides in `TaskStatus`, with `AbstractTask.setStatus(...)` throwing `IllegalArgumentException` on invalid transitions.
- **Extensibility hook:** `TaskManager.registerFactory(...)` enables adding new task types without modifying the coordinator.

Tools and Techniques

- **JDK 21** (bundled with the Red Hat Java extension).
- **VS Code** with Red Hat Java / Debugger / Test Runner extensions.
- **PlantUML + Mermaid** for diagram generation.
- **Git / GitHub** for version control ([hoop-ai/software-architecture-project](https://github.com/hoop-ai/software-architecture-project)).
- **Manual test harness** (`Main.java`) structured into six labeled sections producing PASS/FAIL output.

3. PROJECT MANAGEMENT

3.1 Work Plan

WP	Name	Deliverables	Status
WP1	Problem Definition	Problem statement, functional & non-functional requirements, scope document	Complete
WP2	Design & UML	Seven PlantUML diagrams, design spec (docs/design/design-spec.md), pattern selection rationale	Complete
WP3	Implementation	16 Java source files implementing Factory Method, Strategy, state machine, and <code>TaskManager</code> coordinator	Complete
WP4	Testing	Six test sections in <code>Main.java</code> covering pattern behavior, state transitions, integration, and SOLID compliance; test documentation at docs/design/test-documentation.md	Complete
WP5	Reporting	Full report (docs/report/report.md), presentation outline, submission package	Complete

3.2 Risk Management

Risk	Likelihood	Impact	Mitigation
Over-engineering (applying patterns where simpler code would suffice)	Medium	Medium	Keep scope minimal; every pattern use must be justified by a concrete extensibility requirement.
Pattern misapplication (e.g., confusing Factory Method with Abstract Factory)	Medium	High	Follow GoF definitions strictly; validate with Refactoring Guru and <i>Head First Design Patterns</i> references.
UML diagrams drifting from code	High	Medium	Generate diagrams from PlantUML sources checked into the repo; update diagrams whenever class structure changes.
Java environment/JDK path issues on target machine	Medium	Low	Document build/run commands in the project README and use the JDK bundled with the editor's Java extension.
Scope creep toward persistence or GUI	Medium	High	Explicitly exclude these from scope; record them under "Potential Improvements".
Loss of work / accidental file deletion	Low	High	Git commits after every work package; remote backup on GitHub.
Deadline slippage (June 5, 2026)	Low	High	Milestones per WP; reporting (WP5) started in parallel with implementation (WP3).

4. IMPLEMENTATION

Class Structure

16 Java files in `src/main/java/`: 2 interfaces, 1 enum, 1 abstract class, 12 concrete classes (including `Main`), three concrete tasks, three concrete factories, three concrete strategies, and `TaskManager`).

Interfaces and Abstract Classes

- **Task interface** — the universal contract: `getId`, `getTitle`, `getDescription`, `getStatus` / `setStatus`, `getPriority`, `getDeadline` / `setDeadline`, `getType`, `getCreatedAt`. Deliberately lean to satisfy Interface Segregation.
- **AbstractTask abstract class** — skeletal implementation managing shared fields, auto-incrementing ID counter, state transition validation, and a `toString()` that concrete subclasses extend via `super.toString()`.
- **TaskFactory abstract class** — declares the factory method `createTask(title, description, priority)` and provides the template method `createTaskWithDeadline(...)`.
- **PriorityStrategy interface** — single method `List<Task> sort(List<Task> tasks)`.
- **TaskStatus enum** — per-constant `allowedTransitions()` overrides encoding the finite state machine.

Pattern Implementation

Factory Method. `TaskManager` maintains a `Map<String, TaskFactory>` registry pre-populated with the three concrete factories. `createTask(type, ...)` looks up the factory by type string and delegates. New task types are added via `registerFactory(...)` — no modification to existing classes required.

```
public Task createTask(String type, String title, String description, int priority) {
    TaskFactory factory = factoryRegistry.get(type.toUpperCase());
    if (factory == null) {
        throw new IllegalArgumentException("No factory registered for type: " + type);
    }
    Task task = factory.createTask(title, description, priority);
    tasks.add(task);
    return task;
}
```

Strategy. `TaskManager` holds a `PriorityStrategy` reference (default: `UrgentFirstStrategy`). `getPrioritizedTasks()` delegates to `strategy.sort(tasks)`. `setPriorityStrategy(...)` swaps the algorithm at runtime.

Key Methods and Logic

- `AbstractTask.setStatus(...)` — validates transitions against the `TaskStatus` state machine.
- `BugTaskFactory.createTask(...)` — builds a `BugTask` with default severity "MEDIUM".

- `SeverityFirstStrategy.sort(...)` — two-tier sort: bug tasks first by severity rank (CRITICAL=4, HIGH=3, MEDIUM=2, LOW=1), then non-bug tasks by priority descending. Uses `instanceof BugTask` as the partitioning predicate.
- `TaskManager.transitionTask(id, targetStatus)` — lifecycle management delegated to the state machine in `TaskStatus`.
- `TaskManager.registerFactory(type, factory)` — runtime extensibility hook.

See [docs/report/report.md §3.5](#) for the complete code walkthrough.

5. TESTING AND DEMONSTRATION

All tests are executed by running `Main.java`, which contains six labeled sections. Full sample output is captured in [docs/design/test-output.txt](#).

Test Cases

#	Section	Validates
1	Factory Method Demo	Polymorphic creation through the <code>TaskFactory</code> reference; template method <code>createTaskWithDeadline(...)</code> .
2	Strategy Demo	Same task list sorted three different ways by swapping strategies at runtime.
3	Lifecycle / State Transitions	Valid path OPEN→IN_PROGRESS→REVIEW→DONE; blocked path; invalid-transition rejection; terminal-state enforcement.
4	TaskManager Integration	Mixed task creation, status transitions, filtering by status, summary report, task removal.
5	SOLID Principles Demo	OCP (custom anonymous strategy added at runtime), LSP (factories substitutable), DIP (only abstractions used), SRP, ISP.
6	Interactive Console	Menu-driven CLI for hands-on demonstration.

Sample Inputs/Outputs

Strategy pattern (Test 2):

```
--- Strategy 1: UrgentFirstStrategy (highest priority first) ---
1. Task[...priority=5...] Fix payment bug
2. Task[...priority=4...] Export CSV
3. Task[...priority=3...] Setup guide
4. Task[...priority=2...] Add search
5. Task[...priority=1...] UI glitch

--- Strategy 2: DeadlineFirstStrategy (earliest deadline first) ---
1. Task[...deadline=2026-04-20...] Setup guide
2. Task[...deadline=2026-05-01...] Fix payment bug
3. Task[...deadline=2026-05-10...] Export CSV
4. Task[...deadline=2026-06-15...] Add search
5. Task[...deadline=none...] UI glitch
```

[PASS] Same tasks, three different orderings via Strategy swap.

State machine rejection (Test 3):

```
--- Invalid transition: OPEN -> DONE (should fail) ---
Caught expected error: Cannot transition from OPEN to DONE
[PASS] State machine correctly rejects invalid transitions.
```

Main.java Execution

```
javac -d bin src/main/java/*.java
java -cp bin Main
```

Each section produces human-readable headers, tabular task output, and [PASS] markers, making the test run suitable for live classroom demonstration.

6. RESULTS AND EVALUATION

Advantages of the Design Patterns Used

1. **Separation of concerns** — creation, prioritization, lifecycle, and coordination each live in dedicated classes.
2. **Double extensibility** — new task types *and* new prioritization algorithms can be added independently, each through pure addition.
3. **Runtime flexibility** — strategies and factories are both swappable/registerable at runtime.
4. **SOLID compliance** — all five principles are exercised by the design and demonstrated explicitly in Test 5.
5. **Clean abstractions** — `Task`, `TaskFactory`, and `PriorityStrategy` provide clean boundaries that prevent implementation details from leaking.

Limitations

1. **No persistence** — all state lives in memory.
2. **No GUI** — console application only.
3. **Single-user** — no concurrency; `idCounter` is not thread-safe.
4. **No authentication/authorization** — any caller can perform any operation.
5. **String-based severity** — `BugTask.severity` is a `String` rather than a dedicated enum.

Possible Improvements

1. **Observer Pattern** for notifying team members on status changes.
2. **Command Pattern** for undo/redo of task operations.
3. **Repository Pattern** with database/file backends behind a common interface.
4. **State Pattern** replacing the enum for state-specific behavior beyond transitions.
5. **Builder Pattern** for tasks with many optional parameters.

Alternative Solutions

- **Abstract Factory** instead of Factory Method — appropriate if each task type came with a *family* of related objects (template, validator, renderer). Overkill for this project.
- **Enum-only state representation** — the current approach, sufficient because states carry no behavior beyond transition rules.
- **Simple if/else dispatch** — rejected because it violates OCP and scales poorly.

Justification of Design Pattern Usage

Why Factory Method was necessary. Each task type has distinct construction requirements (bug: severity + repro steps; feature: effort + business value; doc: document type + audience). Without a factory abstraction, every place that creates tasks would need conditional logic over task type, duplicating knowledge across the codebase. Factory Method centralizes creation in one class per type and eliminates direct coupling between `TaskManager` and concrete task classes. It also enables runtime registration of new task types via `registerFactory(...)`, which would be impossible with hard-coded constructors.

Why Strategy was necessary. Prioritization is fundamentally context-dependent — crisis mode demands urgent-first, sprint planning demands deadline-first, release stabilization demands severity-first. These are different algorithms operating on the same data. Embedding them in `TaskManager` with `if/else` branches would bloat the class, violate SRP, and make adding a new algorithm a modification rather than an extension. Strategy isolates each algorithm, enables runtime swapping, and lets `TaskManager` focus purely on coordination.

Counterfactual Analysis — What If the Patterns Were Not Used?

Without Factory Method:

- **Tight coupling:** `TaskManager.createTask(...)` would reference `BugTask`, `FeatureTask`, and `DocumentationTask` directly, pulling concrete classes into a high-level module and violating the Dependency Inversion Principle.
- **Increased code complexity:** A long `switch` on `type` string would need to duplicate default-value logic, validation, and construction for every task type.
- **OCF violation:** Adding a `ResearchTask` would require editing `TaskManager`, editing every place that enumerates task types, and re-testing all existing creation paths — a change that should be purely additive becomes invasive.
- **Reduced testability:** Mocking task creation for unit tests would require either reflection or subclassing `TaskManager`, because the `new` keywords would be hard-wired.

Without Strategy:

- **Monolithic sorting logic:** `TaskManager` would contain three (or more) private methods `sortByUrgency`, `sortByDeadline`, `sortBySeverity`, each with their own comparator logic, plus a `sortMode` field and a dispatching `switch`.
- **SRP violation:** `TaskManager` would be simultaneously responsible for coordination *and* for implementing every sorting algorithm.
- **Reduced scalability:** Adding a fourth algorithm means editing the central class, increasing the blast radius of every future change.
- **Difficulty in extending the system:** A user-supplied sorting rule (e.g., the anonymous "lowest-first" strategy demonstrated in Test 5) could not be injected at runtime — the set of algorithms would be fixed at compile time.
- **Reduced maintainability:** Each sorting method would mix its algorithm with `TaskManager`'s other responsibilities, making unit testing of individual algorithms awkward and encouraging regressions when one algorithm is changed.

In both counterfactuals, the core issue is the same: structural decisions that *should* be open to extension become closed. The Factory Method and Strategy patterns directly neutralize these risks by design.

7. CONCLUSION

Achievements

This project successfully designed, implemented, documented, and tested a Task Management System that composes two GoF design patterns — Factory Method and Strategy — in pure Java with zero external dependencies. The final deliverable comprises 16 Java source files, seven UML diagrams, a full written report, a structured test suite with six sections, and an interactive console application. Every SOLID principle is explicitly exercised by the tests, and both patterns are shown to coexist naturally inside a single coordinator class.

Lessons Learned

1. **Patterns are a vocabulary.** They communicate design intent far more efficiently than ad-hoc descriptions.
2. **SOLID enables extensibility in practice, not just theory.** Throughout implementation, new classes were added by pure extension — no existing class ever had to be opened for modification after its initial creation.
3. **Patterns compose.** Factory Method and Strategy address orthogonal concerns and live comfortably in the same system without interference.
4. **Simplicity has architectural value.** Pure Java, no frameworks, 16 files — and the system is still fully extensible. Good architecture comes from design decisions, not from tools.
5. **UML earns its place when it stays synchronized with code.** PlantUML sources checked into the repo prevent diagrams from drifting.

Importance of Design Patterns

Design patterns are a cornerstone of professional software engineering. They appear in every major framework (Spring, Android, JavaFX), every enterprise codebase, and every technical interview. Mastery of patterns — combined with SOLID principles — marks the transition from programmer to software architect: someone who can design systems that not only work today but remain maintainable and extensible for years. The Factory Method and Strategy patterns demonstrated here are among the most widely used in industry, and the skills practiced in this project transfer directly to real-world software development.

8. REFERENCES

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
2. Freeman, E., & Robson, E. (2020). *Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software* (2nd ed.). O'Reilly Media.
3. Martin, R. C. (2003). *Agile Software Development, Principles, Patterns, and Practices*. Pearson Education.
4. Martin, R. C. (2009). *Clean Code: A Handbook of Agile Software Craftsmanship*. Pearson Education.
5. Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.
6. Oracle. *Java SE Documentation*. <https://docs.oracle.com/javase/>
7. Refactoring Guru. *Design Patterns*. <https://refactoring.guru/design-patterns>
8. Object Management Group. *Unified Modeling Language (UML) Specification*. <https://www.omg.org/spec/UML/>